Smart University Campus Management System Lab Examination

Project 4 Questions and Model Solutions

Complete five phases: write Phases 1–4 on paper and implement Phase 5 in Java. This document contains questions without marks followed by an instructor model solution. Other technically justified designs are acceptable.

Part A Examination Questions

Project scenario

An integrated university platform manages student information, registration, classrooms, schedules, security, notices, and digital services. For the examination, implement account creation, course registration, student evaluation, targeted result notifications, and a simulated external library lookup. Classroom scheduling, RFID attendance, transport, and security monitoring remain extensions.

General instructions

- Use a Java console application with simulated data. A graphical interface, live services, physical devices, and a persistent database are not required.
- Apply Singleton, Factory, Observer, Strategy, and Adapter. Keep class names consistent across analysis, CRC cards, UML, and code.
- State assumptions and distinguish the full platform from the implemented subsystem.

Phase 1 Requirement Analysis

Question 1 Analyze the requirements

- Describe the background and three problems to be addressed.
- State one primary objective and four specific objectives.
- Identify at least four actors, explain their interactions, and define the system boundary.
- Specify four functional and two non-functional requirements linked to the implemented use cases.

Phase 2 Object Identification

Question 2 Classify the objects

Identify at least five entity classes with two or more attributes each, three control classes with responsibilities, and three boundary classes with external interactions. Present three tables and distinguish external actors from internal software representations.

Phase 3 CRC Card Development

Question 3 Prepare six CRC cards

For six important classes, list the class name, at least two responsibilities, and collaborators. Include at least one entity, one control, and one boundary class across the cards.

Phase 4 UML Modeling

Question 4 Model structure and interactions

a. Draw a UML class diagram containing at least eight classes or interfaces. Show attributes, operations, visibility, appropriate multiplicities, associations, and interface realizations. Identify pattern participants. Use aggregation or composition only when ownership and lifecycle semantics justify it.

b. Model a faculty request to evaluate validated marks and publish a personal result. Show mark validation, strategy delegation, publication, and recipient filtering. Use alternatives for valid versus invalid marks; invalid marks must stop before result publication.

Phase 5 Java Implementation

Question 5 Implement all five design patterns

Pattern	Required behavior
Singleton	UniversityConfig supplies one shared configuration instance with a six-credit registration limit.
Factory	AccountFactory creates Student, Faculty, or Administrator through UserAccount; reject unsupported roles. A Simple Factory is acceptable.
Observer	NoticePublisher supports subscription and removal; broadcast notices to portals and filter personal results by recipient ID.
Strategy	EvaluationService switches between TheoryEvaluation and LabEvaluation at runtime.
Adapter	LibraryAdapter implements LibraryService, translating text copy counts from ExternalLibrarySystem into availability.

Examination rules

- Maximum registration load is six credits. Reject duplicate registration, full courses, and a registration exceeding the limit. A load of exactly six is allowed.
- Theory score = 0.60 × exam + 0.40 × coursework. Lab score = 0.30 × exam + 0.20 × coursework + 0.50 × project. Each input must be finite and between 0 and 100 inclusive.
- With exam 80, coursework 70, project 90, the theory score is 76 and the lab score is 83. These are examination policies, not an institution’s official grading rules.
- A public notice has no recipient ID and reaches all subscribed portals. A personal result reaches only the matching student portal; the example faculty portal displays public notices only.
- The simulated library returns BOOK1 → "2", BOOK2 → "0", and unknown IDs → "UNKNOWN". The adapter returns true, false, or an explicit error respectively.

Required Demonstrations and Submission

Area	Required demonstration
Configuration and factory	Same configuration instance; all three account types created; unknown role and blank ID rejected.
Registration	Register two three-credit courses; reject a duplicate, full course, and a third three-credit course. Failed registrations leave credit totals unchanged.
Evaluation	Theory gives 76; after switching policy, lab gives 83. Reject −1, 101, and NaN marks.
Observer	Broadcast reaches all portals; result for S1 does not display in S2 or the faculty portal. Removed S2 portal receives no later broadcast.
Library adapter	BOOK1 available; BOOK2 unavailable; unknown book rejected.

Submit handwritten Phases 1–4, Java source files, compilation and execution instructions, and a test table comparing expected with actual results. Use the supplied fictional values consistently.

Part B Model Solutions

The model solution is one coherent design. It implements the examination scope, while some domain classes remain extensions. The complete Java listing uses nested classes to allow compilation from one source file.

Phase 1 Background and Objectives

Assume student information and digital services are handled in separate tools. Three problems are inconsistent registration checks, repeated manual score calculation, and poorly targeted notifications. The examination solution centralizes registration decisions and separates evaluation and integration logic behind interfaces.

Primary objective: Develop an object-oriented campus subsystem that validates registrations, computes scores using interchangeable policies, sends appropriate notifications, and accesses an external library through a stable interface.

Specific objectives

- Create supported account roles through one creation interface.
- Enforce registration capacity, duplication, and credit constraints.
- Calculate and publish scores using a selected evaluation policy.
- Integrate library availability and distinguish public notices from personal results.

Phase 1 Actors and Requirements

Actor	Interaction
Student	Registers for courses, receives notices and personal results, and checks book availability.
Faculty Member	Supplies assessment marks and chooses the appropriate evaluation policy.
Administrator	Creates account roles and maintains campus configuration and course information.
Librarian	Maintains the external library catalogue used through an adapter.

System boundary: the platform contains domain records, controllers, portals, and adapters. External people, third-party systems, and physical devices lie outside it. A portal or adapter is an internal boundary component, not the external actor itself.

Security Officer and IoT System are broader-platform actors for access monitoring and attendance. The model does not implement their hardware interactions.

Functional requirements

- FR1: Create Student, Faculty, and Administrator accounts through UserAccount and reject unsupported roles.
- FR2: Register a student only when the course is not already selected, a seat is available, and the resulting load is at most six credits.
- FR3: Validate assessment marks, compute scores with the selected policy, and notify the matching student.
- FR4: Broadcast notices, remove subscribers, and translate external library counts into availability.

Non functional requirements

- Maintainability: evaluation algorithms and library-provider integration must be replaceable through interfaces.
- Reliability: rejected registration must not change enrollment or credit totals; invalid marks must not produce a result notification.

Assumptions

One canonical Course object exists for each course code within a registration manager. Student IDs identify the same student across operations. The configuration is immutable, the program is single-threaded, and all data are in memory. Portal setup and faculty actions are simulated by main(). Recipient filtering demonstrates routing; it does not implement authentication or access control.

Phase 2 Object Identification Solution

Entity objects

Entity	Attributes	Implementation scope
Student	id, role	Implemented account and domain representation; role returned by role().
Course	code, credits, capacity, students	Implemented course information and enrolled student IDs.
Marks	exam, coursework, project	Implemented assessment value object.
LibraryBook	isbn, title, availableCopies	Extension; provider exposes copy counts in the example.
Notice	noticeId, message, recipientId	Extension; code passes recipient and message directly.

Control objects

Class	Responsibility
CourseRegistrationManager	Validate and record course registration and credit totals.
EvaluationService	Delegate calculation to the active policy and publish a result.
NoticePublisher	Coordinate registration, removal, and delivery of notification events.

Boundary objects

Class	Interaction
StudentPortal	Display public notices and results matching the student ID.
FacultyPortal	Display public campus notices in this teaching example.
LibraryAdapter	Bridge the incompatible external library query format.

Phase 3 CRC Card Solution

CourseRegistrationManager

Responsibilities: Check duplicate registration, capacity, and credit limit; update enrollment and credit totals.
Collaborators: Student, Course, UniversityConfig

Student

Responsibilities: Retain account identity; report the student role; identify the owner of registrations and results.
Collaborators: UserAccount, AccountFactory, CourseRegistrationManager, EvaluationService

Course

Responsibilities: Retain code, credits, and capacity; retain enrolled student IDs.
Collaborators: CourseRegistrationManager, Student

EvaluationService

Responsibilities: Replace evaluation policy; calculate scores; publish the score for its student.
Collaborators: EvaluationStrategy, Marks, Student, NoticePublisher

NoticePublisher

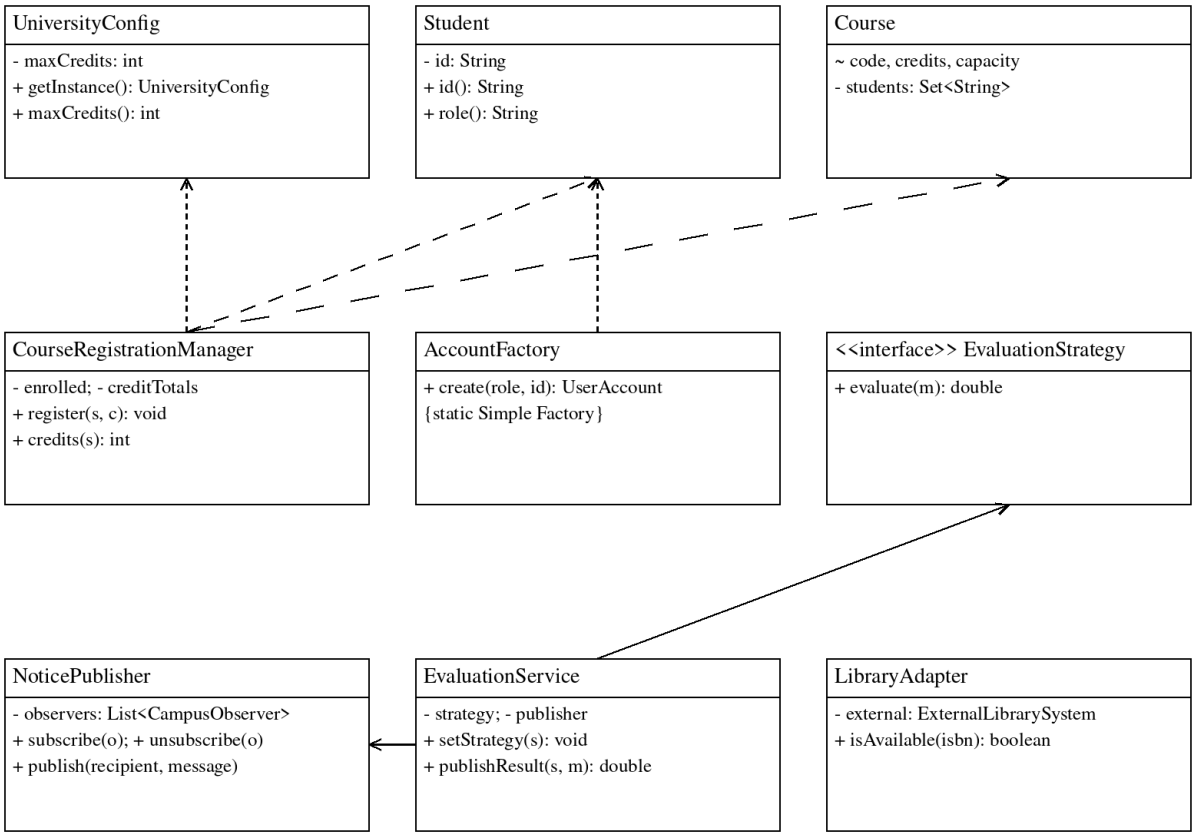
Responsibilities: Maintain subscribers; remove subscribers; deliver recipient-tagged messages.
Collaborators: CampusObserver, StudentPortal, FacultyPortal, EvaluationService

LibraryAdapter

Responsibilities: Query the provider; parse and validate copy counts; return availability or an error.
Collaborators: LibraryService, ExternalLibrarySystem

Phase 4 UML Class Diagram Solution

The diagram shows nine core classes or interfaces. Solid navigable lines indicate retained references and dashed arrows indicate method dependencies. The table completes concrete realizations and multiplicities. Package-visible fields use ~. Constructors and some helper details are omitted.

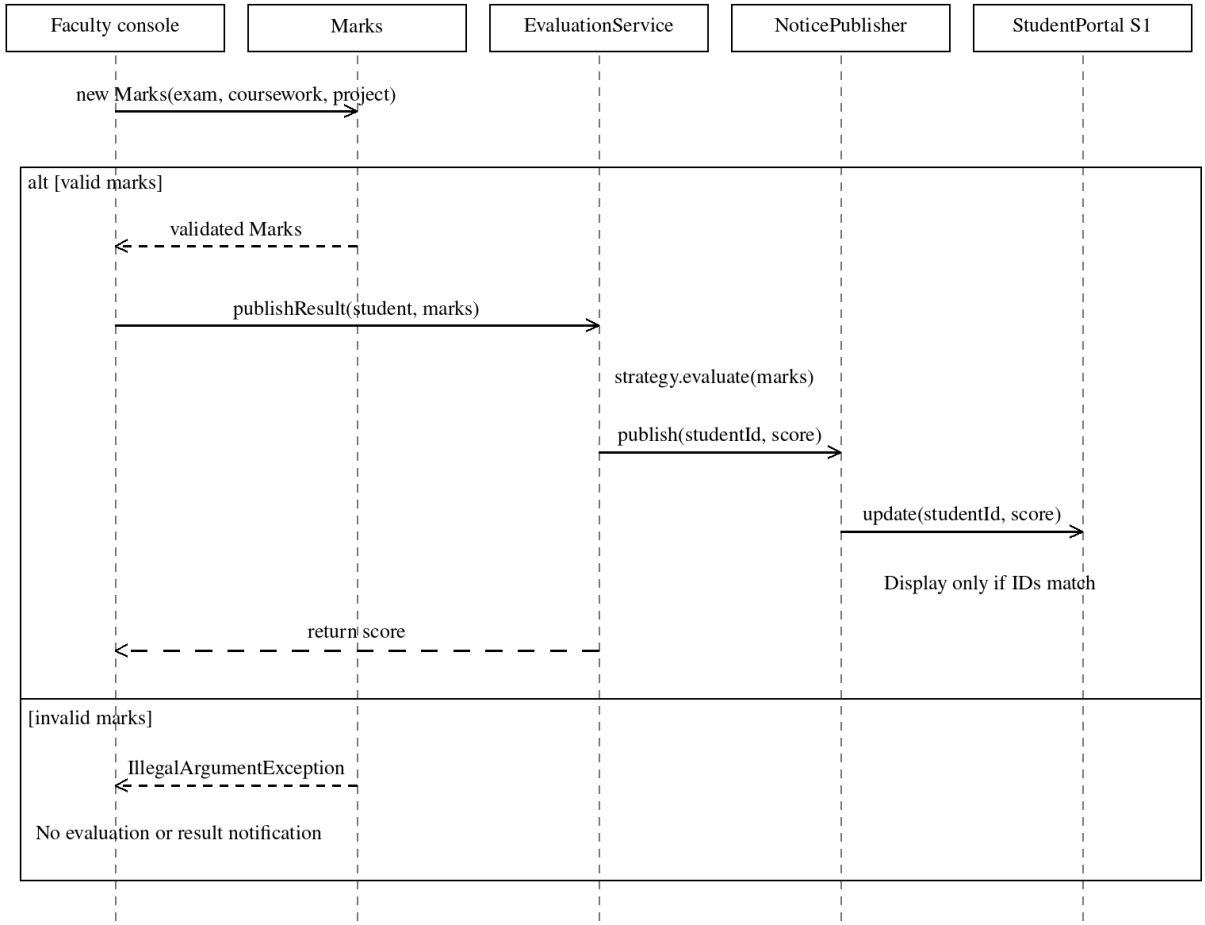


Additional interface realizations and associations are specified in the table below.

Relationship	Details
Factory participants	Student, Faculty, and Administrator realize UserAccount. AccountFactory depends on their constructors.
Registration relationships	One manager stores registrations for 0..* students; one Course records 0..* student IDs bounded by capacity. Each registration connects a student ID and a course code.
Observer participants	StudentPortal and FacultyPortal realize CampusObserver. One NoticePublisher stores 0..* observers; one EvaluationService holds one publisher.
Strategy participants	TheoryEvaluation and LabEvaluation realize EvaluationStrategy. Each EvaluationService holds one active strategy and depends on Marks and Student.
Adapter participants	LibraryAdapter realizes LibraryService and holds one ExternalLibrarySystem. Availability queries are independent of evaluation.

Phase 4 UML Sequence Diagram Solution

Model a faculty request to evaluate validated marks and publish a personal result. Show mark validation, strategy delegation, publication, and recipient filtering. Use alternatives for valid versus invalid marks; invalid marks must stop before result publication.



The faculty console constructs Marks before calling publishResult(). The Marks constructor rejects invalid inputs. EvaluationService delegates to its selected EvaluationStrategy; the diagram compresses this delegation into the calculation note. NoticePublisher calls update(recipient, message) on every registered observer. S1 displays its own result; S2 and the faculty portal filter this personal event. Filtering is an output rule rather than an authorization mechanism.

Associations and dependencies are sufficient for this model; no composition ownership is assumed. Keep the detailed message names aligned with the Java operations and the stated diagram simplifications.

Phase 5 Java Solution Listing 1

Save the complete listing as SUCMSExam.java. Compile with javac SUCMSExam.java and run java SUCMSExam. The code was compiled and executed with OpenJDK 17.0.20; all included checks passed. No third-party libraries are required.

The Singleton applies within this application and class-loader scope. The factory is a Simple Factory. The program is a sequential in-memory teaching simulation. Copy all listing sections in order into one source file.

```
import java.util.*;
public class SUCMSExam {
  static void requireText(String s) {
    if (s == null || s.trim().isEmpty())
      throw new IllegalArgumentException("Blank value");
  }
  static final class UniversityConfig {
    private static final UniversityConfig INSTANCE = new UniversityConfig();
    private final int maxCredits = 6;
    private UniversityConfig() { }
    public static UniversityConfig getInstance() { return INSTANCE; }
    public int maxCredits() { return maxCredits; }
  }
  interface UserAccount { String id(); String role(); }
  static final class Student implements UserAccount {
    private final String id;
    Student(String id) { requireText(id); this.id = id; }
    public String id() { return id; }
    public String role() { return "STUDENT"; }
  }
  static final class Faculty implements UserAccount {
    private final String id;
    Faculty(String id) { requireText(id); this.id = id; }
    public String id() { return id; }
    public String role() { return "FACULTY"; }
  }
  static final class Administrator implements UserAccount {
    private final String id;
```

Phase 5 Java Solution Listing 2

```
  Administrator(String id) { requireText(id); this.id = id; }
  public String id() { return id; }
  public String role() { return "ADMIN"; }
 }
 static final class AccountFactory {
  public static UserAccount create(String role, String id) {
   if ("STUDENT".equals(role)) return new Student(id);
   if ("FACULTY".equals(role)) return new Faculty(id);
   if ("ADMIN".equals(role)) return new Administrator(id);
   throw new IllegalArgumentException("Unsupported role");
  }
 }
 static final class Course {
  final String code;
  final int credits, capacity;
  private final Set<String> students = new HashSet<>();
  Course(String code, int credits, int capacity) {
   requireText(code);
   if (credits <= 0 || capacity <= 0) throw new IllegalArgumentException();
   this.code = code; this.credits = credits; this.capacity = capacity;
  }
 }
 static final class CourseRegistrationManager {
  private final Map<String, Set<String>> enrolled = new HashMap<>();
  private final Map<String, Integer> creditTotals = new HashMap<>();
  public void register(Student s, Course c) {
   Set<String> codes = enrolled.getOrDefault(s.id(), Collections.emptySet());
   int current = creditTotals.getOrDefault(s.id(), 0);
   if (codes.contains(c.code)) throw new IllegalStateException("Duplicate course");
   if (c.students.size() >= c.capacity) throw new IllegalStateException("Full course");
   if (current + c.credits > UniversityConfig.getInstance().maxCredits())
    throw new IllegalStateException("Credit limit");
   enrolled.computeIfAbsent(s.id(), key -> new HashSet<>()).add(c.code);
   c.students.add(s.id()); creditTotals.put(s.id(), current + c.credits);
  }
  public int credits(Student s) { return creditTotals.getOrDefault(s.id(), 0); }
 }
 interface CampusObserver { void update(String recipient, String message); }
 static final class StudentPortal implements CampusObserver {
  private final String studentId;
  int received;
  StudentPortal(String id) { studentId = id; }
  public void update(String recipient, String message) {
```

Phase 5 Java Solution Listing 3

```
    if (recipient != null && !recipient.equals(studentId)) return;
    received++; System.out.println("Student " + studentId + ": " + message);
  }
 }
 static final class FacultyPortal implements CampusObserver {
  int received;
  public void update(String recipient, String message) {
    if (recipient != null) return; // Only public notices in this example.
    received++; System.out.println("Faculty: " + message);
  }
 }
 static final class NoticePublisher {
  private final List<CampusObserver> observers = new ArrayList<>();
  public void subscribe(CampusObserver o) {
    if (!observers.contains(o)) observers.add(o);
  }
  public void unsubscribe(CampusObserver o) { observers.remove(o); }
  public void publish(String recipient, String message) {
    for (CampusObserver o : new ArrayList<>(observers)) o.update(recipient, message);
  }
 }
 static final class Marks {
  final double exam, coursework, project;
  Marks(double exam, double coursework, double project) {
    for (double x : new double[] {exam, coursework, project})
      if (!Double.isFinite(x) || x < 0 || x > 100)
        throw new IllegalArgumentException("Marks must be 0 to 100");
    this.exam = exam; this.coursework = coursework; this.project = project;
  }
 }
 interface EvaluationStrategy { double evaluate(Marks m); }
 static final class TheoryEvaluation implements EvaluationStrategy {
  public double evaluate(Marks m) { return .6 * m.exam + .4 * m.coursework; }
 }
 static final class LabEvaluation implements EvaluationStrategy {
  public double evaluate(Marks m) {
    return .3 * m.exam + .2 * m.coursework + .5 * m.project;
  }
 }
 static final class EvaluationService {
  private EvaluationStrategy strategy;
  private final NoticePublisher publisher;
  EvaluationService(EvaluationStrategy s, NoticePublisher p) {
```

Phase 5 Java Solution Listing 4

```
    strategy = s; publisher = p;
  }
  public void setStrategy(EvaluationStrategy s) { strategy = s; }
  public double publishResult(Student student, Marks marks) {
    double score = strategy.evaluate(marks);
    publisher.publish(student.id(), "Score=" + score); return score;
  }
}
interface LibraryService { boolean isAvailable(String isbn); }
static final class ExternalLibrarySystem {
  public String copiesAsText(String isbn) {
    if ("BOOK1".equals(isbn)) return "2";
    if ("BOOK2".equals(isbn)) return "0";
    return "UNKNOWN";
  }
}
static final class LibraryAdapter implements LibraryService {
  private final ExternalLibrarySystem external;
  LibraryAdapter(ExternalLibrarySystem external) { this.external = external; }
  public boolean isAvailable(String isbn) {
    requireText(isbn);
    String text = external.copiesAsText(isbn);
    if ("UNKNOWN".equals(text)) throw new IllegalArgumentException("Unknown book");
    int copies = Integer.parseInt(text);
    if (copies < 0) throw new IllegalArgumentException("Invalid copies");
    return copies > 0;
  }
}
static void expect(boolean ok, String label) {
  if (!ok) throw new AssertionError(label);
  System.out.println("PASS: " + label);
}
static void rejects(Runnable action, String label) {
  try { action.run(); }
  catch (IllegalArgumentException | IllegalStateException e) {
    System.out.println("PASS: " + label); return;
  }
  throw new AssertionError(label);
}
public static void main(String[] args) {
  expect(UniversityConfig.getInstance() == UniversityConfig.getInstance(), "singleton");
  Student student = (Student) AccountFactory.create("STUDENT", "S1");
  Student second = (Student) AccountFactory.create("STUDENT", "S2");
```

Phase 5 Java Solution Listing 5

```
    expect(AccountFactory.create("FACULTY", "F1") instanceof Faculty, "faculty factory");
    expect(AccountFactory.create("ADMIN", "A1") instanceof Administrator, "admin factory");
    rejects(() -> AccountFactory.create("UNKNOWN", "X"), "unsupported account");
    rejects(() -> AccountFactory.create("STUDENT", ""), "blank account ID");
    CourseRegistrationManager manager = new CourseRegistrationManager();
    Course c1 = new Course("C1", 3, 1), c2 = new Course("C2", 3, 5);
    Course c3 = new Course("C3", 3, 5);
    manager.register(student, c1);
    rejects(() -> manager.register(student, c1), "duplicate registration");
    rejects(() -> manager.register(second, c1), "capacity limit");
    manager.register(student, c2);
    expect(manager.credits(student) == 6, "six credit boundary accepted");
    rejects(() -> manager.register(student, c3), "credit limit exceeded");
    expect(manager.credits(student) == 6 && manager.credits(second) == 0,
        "failed registration leaves credits unchanged");
    NoticePublisher publisher = new NoticePublisher();
    StudentPortal p1 = new StudentPortal("S1"), p2 = new StudentPortal("S2");
    FacultyPortal faculty = new FacultyPortal();
    publisher.subscribe(p1); publisher.subscribe(p2); publisher.subscribe(faculty);
    publisher.publish(null, "Library hours updated");
    expect(p1.received == 1 && p2.received == 1 && faculty.received == 1,
        "broadcast notice");
    EvaluationService evaluation = new EvaluationService(new TheoryEvaluation(), publisher);
    Marks m = new Marks(80, 70, 90);
    expect(Math.abs(evaluation.publishResult(student, m) - 76) < 1e-9, "theory score 76");
    expect(p1.received == 2 && p2.received == 1 && faculty.received == 1,
        "result recipient filtering");
    evaluation.setStrategy(new LabEvaluation());
    expect(Math.abs(evaluation.publishResult(student, m) - 83) < 1e-9, "lab score 83");
    publisher.unsubscribe(p2); publisher.publish(null, "Campus notice");
    expect(p2.received == 1 && p1.received == 4 && faculty.received == 2,
        "observer removal");
    rejects(() -> new Marks(-1, 70, 90), "negative marks rejected");
    rejects(() -> new Marks(101, 70, 90), "marks above 100 rejected");
    rejects(() -> new Marks(Double.NaN, 70, 90), "NaN marks rejected");
    LibraryService library = new LibraryAdapter(new ExternalLibrarySystem());
    expect(library.isAvailable("BOOK1"), "available book");
    expect(!library.isAvailable("BOOK2"), "unavailable book");
    rejects(() -> library.isAvailable("MISSING"), "unknown book");
    System.out.println("All checks completed.");
  }
}
```

Testing Results and Design Interpretation

The main method executes checks and throws AssertionError on a mismatch. Its rejects() helper checks expected application exceptions. The following areas passed in the recorded Java run.

Test area	Observed behavior
Configuration and factory	Same configuration instance; all three account types created; unknown role and blank ID rejected.
Registration	Register two three-credit courses; reject a duplicate, full course, and a third three-credit course. Failed registrations leave credit totals unchanged.
Evaluation	Theory gives 76; after switching policy, lab gives 83. Reject −1, 101, and NaN marks.
Observer	Broadcast reaches all portals; result for S1 does not display in S2 or the faculty portal. Removed S2 portal receives no later broadcast.
Library adapter	BOOK1 available; BOOK2 unavailable; unknown book rejected.

Recorded output excerpts

```
PASS: singleton
PASS: six credit boundary accepted
PASS: failed registration leaves credits unchanged
PASS: theory score 76
PASS: result recipient filtering
PASS: lab score 83
PASS: observer removal
PASS: available book
PASS: unknown book
All checks completed.
```

These excerpts omit additional passing checks and interleaved notification messages. All checks completed. appears only after the earlier checks succeed.

Pattern rationale

The configuration Singleton centralizes an immutable policy. The Simple Factory isolates account construction. Evaluation strategies isolate scoring rules, observers separate publication from portal delivery, and the adapter isolates the library's data representation.

Limitations

The example omits authentication, durable records, prerequisite checking, timetable conflicts, real library loans, transport, RFID hardware, and security-camera processing. The formulas and six-credit cap are fictional examination rules.